

Atividade – Reescrita de Algoritmos em Rust

Obs:

Instruções para executar os códigos da atividade em Rust

1. Instalar o Rust

Se ainda não tiver o Rust instalado, baixe pelo site oficial:

<https://www.rust-lang.org/tools/install>

Após a instalação, verifique se tudo está funcionando:

2. Criar um projeto Cargo

No terminal, execute:

Isso criará a estrutura básica do projeto.

3. Adicionar os arquivos dos exercícios

Dentro da pasta src/, crie os arquivos:

- ex01_verificar_primeiro.rs
- ex02_somar_lista.rs
- ex03_busca_binaria.rs
- ex04_pares_com_soma.rs
- ex05_imprimir_pares_e_pares.rs
- ex06_potencias_de_dois.rs
- ex07_fibonacci_recurso.rs
- ex08_ordenacao_bolha.rs
- ex09_produto_de_matrizes.rs
- ex10_merge_sort.rs

Segue abaixo os exercíscs

Exercício 1 – Verificar Primeiro

Complexidade: $O(1)$

Código em Rust:

```
pub fn verificar_primeiro(lista: &[i32]) -> Option<i32> {
    lista.first().copied()
}
```

Exercício 2 – Somar Lista

Complexidade: $O(n)$

Código em Rust:

```
pub fn somar_lista(lista: &[i32]) -> i32 {
    let mut total = 0;
    for &elemento in lista {
        total += elemento;
    }
    total
}
```

Exercício 3 – Busca Binária

Complexidade: $O(\log n)$

Código em Rust:

```
pub fn busca_binaria(lista: &[i32], alvo: i32) -> Option<usize> {
    let mut esquerda: isize = 0;
    let mut direita: isize = lista.len() as isize - 1;

    while esquerda <= direita {
        let meio = (esquerda + direita) / 2;
        let idx = meio as usize;

        if lista[idx] == alvo {
            return Some(idx);
        } else if lista[idx] < alvo {
            esquerda = meio + 1;
        } else {
            direita = meio - 1;
        }
    }
    None
}
```

Exercício 4 – Pares com Soma

Complexidade: $O(n^2)$

Código em Rust:

```
pub fn pares_com_soma(lista: &[i32], alvo: i32) {
    for i in 0..lista.len() {
        for j in (i + 1)..lista.len() {
            if lista[i] + lista[j] == alvo {
                println!("{}", lista[i], lista[j]);
            }
        }
    }
}
```

Exercício 5 – Imprimir Pares e Pares

Complexidade: $O(n^2)$

Código em Rust:

```
pub fn imprimir_pares_e_pares(lista: &[i32]) {
    for &x in lista {
        println!("{}", x);
    }

    for &x in lista {
        for &y in lista {
            println!("{}", x, y);
        }
    }
}
```

Exercício 6 – Potências de Dois

Complexidade: $O(\log n)$

Código em Rust:

```
pub fn potencias_de_dois(n: u64) {
    let mut i: u64 = 1;

    while i < n {
        println!("{}", i);
        i *= 2;
    }
}
```

Exercício 7 – Fibonacci Recursivo

Complexidade: $O(2^n)$

Código em Rust:

```
pub fn fibonacci_recursivo(n: u64) -> u64 {
    if n <= 1 {
        return n;
    }
    fibonacci_recursivo(n - 1) + fibonacci_recursivo(n - 2)
}
```

Exercício 8 – Ordenação Bolha

Complexidade: $O(n^2)$

Código em Rust:

```
pub fn ordenacao_bolha(lista: &mut [i32]) {
    let n = lista.len();

    for i in 0..n {
        for j in 0..(n - i - 1) {
            if lista[j] > lista[j + 1] {
                lista.swap(j, j + 1);
            }
        }
    }
}
```

Exercício 9 – Produto de Matrizes

Complexidade: $O(n^3)$

Código em Rust:

```
pub fn produto_de_matrizes(a: &[Vec<i64>], b: &[Vec<i64>]) -> Vec<Vec<i64>> {
    let n = a.len();
    let mut c = vec![vec![0i64; n]; n];

    for i in 0..n {
        for j in 0..n {
            for k in 0..n {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }
    c
}
```

Exercício 10 – Merge Sort

Complexidade: $O(n \log n)$

Código em Rust:

```
pub fn merge_sort(lista: Vec<i32>) -> Vec<i32> {  
    if lista.len() <= 1 {  
        return lista;  
    }  
  
    let meio = lista.len() / 2;  
    let esquerda = merge_sort(lista[..meio].to_vec());  
    let direita = merge_sort(lista[meio..].to_vec());  
  
    merge(esquerda, direita)  
}
```